

Week 5: Markov models and Tensor Backpropagation

<https://mlvu.github.io>

March 8, 2023

1 Markov model classification

This exercise is a follow-up to the naive Bayes classifier of the 3rd week. We recommend briefly reviewing that one. This exercise will be much easier if you do so.

In the lecture, the Bayes sequence classification was explained. The result of training is the frequency of words in different classes. Assume that we train an email spam classifier, and Table 1 shows the frequency of the words in spam and ham emails with a **first-order Markov model**. In total, the number of words in **spam** and **ham** emails is **20,000** and **100,000**, respectively.

Table 1: Frequency of the words in spam and ham emails.

bigrams & unigrams	frequency	
	spam	ham
claim	2000	1000
prize	5000	1000
you	2000	2000
won	2000	1000
claim prize	300	200
prize you	500	100
you won	800	200

We receive the following four-word email.

email: claim prize you won

To classify this email, we are going to use the *chain rule of probability* to simplify the conditional probability $p(\text{claim, prize, you, won} \mid \text{spam})$.

question 1: If we are ultimately interested in the probability $p(\text{spam} \mid \text{claim, prize, you, won})$, then how do we end up with the probability above?

question 2: Take the value $p(\text{claim, prize, you, won} \mid \text{spam})$ and apply the chain rule first (so that every word becomes conditioned on the words before it). Then, apply the Markov assumption, with order 1.

question 3: To fill in the rest of Bayes' rule, we also need the prior probability $p(\text{spam})$, that an email is spam. Give two ways we might estimate this in real life?

Assume we have two priors. Prior 1 says that emails are three times as likely to be ham as spam. Prior 2 says that the probability of a spam email is $1/40$.

question 4: Under prior 1, is the email above more likely to be spam or ham?

question 5: Under prior 2, is the email above more likely to be spam or ham?

question 6: Compute the full class probabilities under prior 2.

One way of expressing the certainty of the classification ham is by computing the probability ratio of ham over spam, or the *odds* of the class ham (according to the classifier).

question 7: Using prior 1, compute the *probability ratio* of spam over ham. What does it tell you?

1.1 Extra practice

We are given a dataset of email, labeled spam and ham, where the number of spam emails is 1% of the total. The total number of words in the spam and ham datasets is 50 000 and 60 000, respectively.

On this dataset, we want to train a *first-order Markov model* as a basis for a Bayes classifier to detect spam.

The following table shows the frequency of several bigrams and unigrams in the dataset.

uni- and bigrams	frequency	
	spam	ham
congratulations	4 000	1 000
you	5 000	5 000
won	8 000	3 000
bitcoin	2 000	2 000
lottery	3 000	500
congratulations you	1 000	200
you won	1 000	500
won bitcoin	1 000	100
bitcoin lottery	800	100
won lottery	500	100
lottery congratulations	40	50

Consider two emails with the following contents:

email 1:	congratulations you won bitcoin lottery
email 2:	you won bitcoin lottery congratulations

Use the data to estimate the prior probabilities of **ham** and **spam**.

question 8: How are the two emails classified?

- A Both are **spam**.
- B The first is **spam**, the second is **ham**.
- C The first is **ham**, the second is **spam**.
- D Both are **ham**.

question 9: What is the probability that the first email is **spam**?

- A $\frac{11}{51}$
- B $\frac{40}{51}$
- C $\frac{99}{199}$
- D $\frac{100}{199}$

question 10: What is the *probability ratio* of **spam** over **ham** for the second email?

- A $\frac{32}{330}$
- B $\frac{330}{32}$
- C $\frac{37}{99}$
- D $\frac{99}{37}$

2 Backpropagation Revisited

2.1 The multivariate chain rule

If we want to do backpropagation for a computation graph where an output variable depends on an input variable through multiple intermediate values (the graph contains a diamond), we require the *multivariate chain rule*. If you don't quite remember how it goes, re-read slides 24–29 of the deep learning lecture before trying these questions.

We will use the backpropagation algorithm to find the derivative of the function

$$f(x) = \sin(x^2) \cos(x^3)$$

with respect to x .

question 11: First, we break the function up into modules. Fill in the blanks.

$$f = a b$$

$$a = \dots$$

$$b = \dots$$

$$c = x^2$$

$$d = x^3$$

question 12: Draw the computation graph with nodes x, c, d, a, b, f

question 13: Work out the *local* derivatives. Which is the correct expression for the gradient $\partial f / \partial x$?

This is a common exam question so make sure to practice if you're not sure. You can easily create new questions for yourself by coming up with any function $f(x)$ with a diamond shape in the computation (just make sure that x is used twice).

2.2 Tensor Backpropagation

In scalar backpropagation, we need to work out only the local derivatives. Once we have these, we can multiply them in any order to get the global

derivative. If we want to apply backpropagation to tensors, things are not so easy.

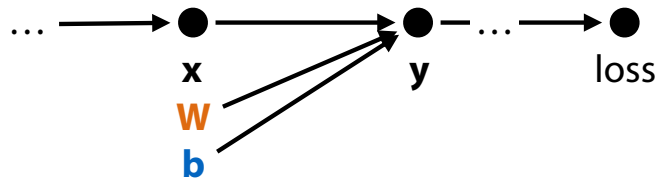
question 14: Why not? What is it about the local derivatives in tensor backpropagation that makes it difficult to apply in this way?

The solution is not to compute the local derivatives explicitly, but only to compute the gradients on the inputs over the loss, given the gradients of the outputs over the loss.

We will practice this for a simple feedforward layer (without activation). Consider a module f that computes:¹

$$\mathbf{y} = f(\mathbf{x}, \mathbf{W}, \mathbf{b}) = \mathbf{W}\mathbf{x} + \mathbf{b}$$

Assume that this module is part of a much larger network. Its output $\mathbf{y}$ is fed as input to another module, which produces a new output that is fed to another module and so on. At the end, a single scalar loss L is produced.



Ultimately, we want to work out the derivative of this loss, with respect to our inputs:

$$\mathbf{x}^\nabla, \mathbf{W}^\nabla, \mathbf{b}^\nabla$$

Where the notation $\mathbf{W}^\nabla$ represents a matrix for the scalar derivatives of the loss with respect to elements of $\mathbf{W}$. For instance, element (2, 3) of matrix $\mathbf{W}^\nabla$ is the scalar derivative $\partial l / \partial W_{23}$.

We'll start with the bias term $\mathbf{b}$. Since matrix/vector calculus can get complicated, and doesn't translate to tensors of higher rank, we'll develop everything in terms of scalar calculus. Once we've taken the derivatives we'll transform everything to matrix operations to make the implementation efficient.

We're interested in $\frac{\partial L}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{b}_j}$, but we need matrix/vector calculus to do that. Instead we will consider $f(\mathbf{b})$ as a *scalar* computation, which has multiple intermediate values $y_1, y_2 \dots, y_k$. By the multivariate chain rule,

¹Normally, we would consider $\mathbf{x}$ the input, and $\mathbf{W}$ and $\mathbf{b}$ the parameters, but for our AD engine, the distinction does not matter: everything going in to the computation is an input, and we may need the gradient over all three.

we can just take the derivative over each path (through each value y_i), and sum them:

$$\frac{\partial l}{\partial \mathbf{b}_j} = \sum_i \frac{\partial l}{\partial y_i} \frac{\partial y_i}{\partial \mathbf{b}_j} = \sum_i y_i^\nabla \frac{\partial y_i}{\partial \mathbf{b}_j}$$

We will assume that $\mathbf{y}^\nabla$ is given to us by the automatic differentiation (AD) engine as a vector. All we need to work out is a simple matrix operation that computes $\partial l / \partial \mathbf{b}_j$ for all j and lays the results out in the same shape as $\mathbf{b}$. We'll start by working out the scalar derivative.

question 15: Fill in the gaps

$$\begin{aligned} \frac{\partial l}{\partial \mathbf{b}_j} &= \sum_i \frac{\partial l}{\partial y_i} \frac{\partial y_i}{\partial \mathbf{b}_j} = \sum_i y_i^\nabla \dots \\ &= \sum_i y_i^\nabla \frac{\partial [\dots]_i}{\partial \mathbf{b}_j} = \sum_i y_i^\nabla \frac{\partial [\mathbf{W}\mathbf{x}]_i + \mathbf{b}_i}{\partial \mathbf{b}_j} \\ &= \sum_i y_i^\nabla \frac{\partial \mathbf{b}_i}{\partial \mathbf{b}_j} = \dots \end{aligned}$$

question 16: The `backward()` function for f if given $\mathbf{y}^\nabla$. What should it return as the gradient for $\mathbf{b}$?

We'll do the same for the gradient over $\mathbf{x}$.

question 17: Work out the scalar derivative $\partial l / \partial x_j$ using the multivariate chain rule (again taking $y_i^\nabla = \partial l / \partial y_i$ as given).

question 18: What should the `backward()` function for f return as the gradient for $\mathbf{x}$?